

TECHNICAL WHITEPAPER · V2.0

People Powered Politics

Decentralised Civic Participation on Cardano & Midnight

Version	1.0 — Public draft for developer review
Date	2025
Audience	Blockchain engineers · Midnight developers · SSI architects · Privacy engineers
Stack	Cardano mainnet · Midnight sidechain · Hyperledger Identus · Compact · Aiken/Plutus
Repo	github.com/SBGoodGit/P3
Contact	p3democracy.io @P3Democracy

Abstract

P3 (People Powered Politics) is a privacy-first civic participation platform built on a dual-layer blockchain architecture. Midnight handles the private compute layer — identity verification, vote confidentiality, and ZKP-based eligibility proofs. Cardano mainnet provides the public settlement and immutable audit anchor layer. Self-Sovereign Identity via Hyperledger Identus decouples voter eligibility from vote registration. A secondary forum layer — the Parliament 3 — enables citizens to originate policy proposals that travel via smart contract to government. This document specifies the full technical architecture, smart contract design, data flows, privacy model, and integration surface for developers building on or contributing to the P3 platform.

1. System Architecture Overview

P3 operates across three distinct layers, each with a clearly defined responsibility. No single layer handles both identity and vote data — the separation is architectural, not a policy rule.

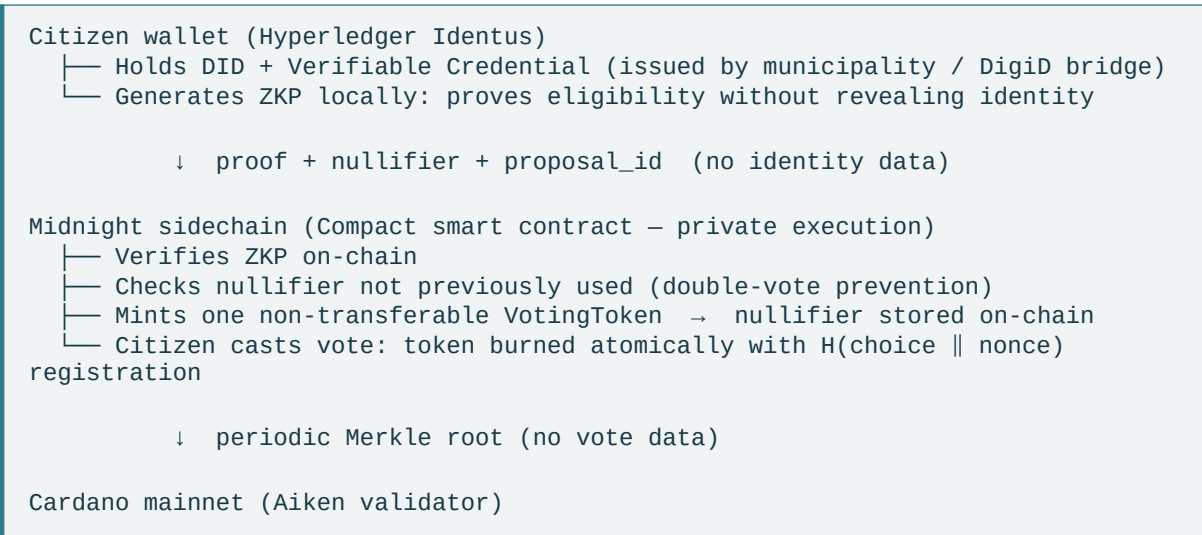
Layer	Technology	Responsibility
Layer 1 — Identity	Hyperledger Indentus (SSI)	Issues and verifies Verifiable Credentials. Citizen wallet stores DID + VC locally. P3 never holds identity data.
Layer 2 — Private Compute	Midnight sidechain (Compact)	ZKP-based eligibility verification. Mint-and-burn of VotingTokens. Private smart contract execution. Parliament 3 forum contracts.
Layer 3 — Public Anchor	Cardano mainnet (Aiken)	Periodic Merkle root anchoring. Immutable public audit trail. Verifiable by any external party.

1.1 Design Principles

- Privacy by architecture, not policy — no personal data ever reaches the blockchain.
- Separation of concerns — identity verification, vote registration, and public audit are strictly isolated layers.
- One citizen, one token, one vote — enforced cryptographically via mint-and-burn on Midnight.
- Auditability without identifiability — Merkle anchoring on Cardano enables external audit without exposing individual votes.
- Modularity — each layer can be upgraded independently without affecting the others.

1.2 Complete Vote Lifecycle

The full lifecycle of a single vote, from citizen wallet to public anchor:



- └─ Batchers submit Merkle root as metadata transaction
- └─ Anchor stored immutably on mainnet
- └─ External auditors: verify sidechain integrity against mainnet anchors

2. Identity Layer — Hyperledger Identus & SSI

The identity layer is the only point in the system where a citizen's real-world identity is involved. Everything downstream operates on cryptographic proofs derived from that identity — never on the identity itself.

2.1 Self-Sovereign Identity Trust Triangle

- A trusted authority (municipality, DigiD bridge, or eIDAS 2.0-compliant provider) that issues a VC asserting citizen attributes. Issuer:
- The citizen. DID and VCs are stored exclusively in the Identus wallet on the citizen's own device. P3 has zero access to this wallet. Holder:
- The P3 Midnight smart contract, which receives a Zero-Knowledge Proof derived from the VC — not the VC itself. Verifier:

2.2 Verifiable Credential Schema (minimum)

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1"],
  "type": ["VerifiableCredential", "P3EligibilityCredential"],
  "issuer": "did:prism:<municipality_did>",
  "credentialSubject": {
    "id": "did:prism:<citizen_did>",
    "eligible": true,
    "scope": "municipality:<municipality_id>",
    "ageAtLeast": 18
  },
  "proof": { ... } // issuer signature (BBS+ or Ed25519)
}
```

2.3 ZKP Proof Specification

For each vote, the citizen wallet generates a proof π satisfying the following statement, using the Identus SDK ZKP module (BBS+ selective disclosure or Groth16 — see Section 7 for open design question):

```
// Public inputs (transmitted to Midnight contract):
nullifier      = H(citizen_secret || proposal_id) // unique per citizen per
proposal
proposal_id    // identifies the vote round
eligibility_root // Merkle root of trusted issuer DIDs (published by P3)

// Private inputs (never leave the citizen device):
citizen DID private key
VC contents (all claims)
citizen_secret (random, device-local)

// Statement proved:
 $\pi$  proves that the holder:
(1) controls a DID whose VC was issued by an issuer in eligibility_root
```

```
(2) VC asserts eligible=true, scope>=proposal.municipality_id, ageAtLeast>=18,
not expired
(3) nullifier = H(citizen_secret || proposal_id) – binding to this specific
proposal
```

The nullifier is stored on-chain after token minting. Any subsequent attempt using the same `citizen_secret` for the same proposal produces an identical nullifier — rejected by the contract. This is the primary double-vote prevention mechanism.

2.4 SDK Integration (TypeScript)

```
// Identus SDK – citizen app (simplified)

// One-time setup (first app launch)
const did = await identus.createDID({ method: 'prism' })

// Request VC from municipality issuer
const vc = await identus.requestCredential({
  issuerDID: MUNICIPALITY_DID,
  claims: { eligible: true, scope: 'municipality:NL-0363', ageAtLeast: 18 }
})

// Per-vote: generate ZKP (runs locally, no network call)
const { proof, nullifier } = await identus.generateZKP({
  credential: vc,
  disclosures: ['eligible', 'scope'], // selective disclosure
  context: { proposal_id: PROPOSAL_ID, eligibility_root: ROOT_HASH }
})

// Submit to P3 API – no identity data in this call
const { token_ref } = await p3api.post('/eligibility/verify', {
  proof, nullifier, proposal_id: PROPOSAL_ID
})
```

3. Private Compute Layer — Midnight & Compact

Midnight is Cardano's privacy-preserving partner chain, designed for confidential smart contract execution. P3 uses Midnight as its primary execution environment for all vote-sensitive operations. Contracts are written in Compact, Midnight's domain-specific language for private circuits.

3.1 Why Midnight

Reason	Implication for P3
Private contract execution	Vote choices and eligibility proofs are processed in a shielded execution environment — not visible on any public ledger or to any node operator.
Native ZKP verification	Midnight's runtime natively verifies ZKP proofs, eliminating the need for external verifier contracts or trusted setup ceremonies at the application level.
Cardano interoperability	Midnight is an official Cardano partner chain. Merkle roots bridge to mainnet via the batcher without complex cross-chain infrastructure.
Compact language	Type-safe, formally verifiable smart contract language. Reduces attack surface for vote integrity bugs.
DUST token model	Midnight's native token (DUST) handles gas fees, keeping VotingTokens purely functional and non-monetary.

3.2 VotingToken Contract — Compact Pseudocode

```
// P3 VotingToken – Compact contract (representative specification)

contract VotingToken {

  // Private state – not visible outside shielded execution
  state nullifiers: Set<Hash>           // spent nullifiers
  state vote_hashes: List<Hash>         // all registered vote hashes (for
Merkle)
  state proposalActive: Bool
  state proposal_id: Bytes32
  state eligibility_root: Hash           // Merkle root of trusted issuer DIDs

  // MINT – called once per eligible citizen per proposal
  @private
  action mint(
    proof: ZKProof,
    nullifier: Hash,
    claimed_root: Hash
  ): VotingToken {
```

```

    assert proposalActive == true
    assert claimed_root == self.eligibility_root      // correct issuer set
    assert !nullifiers.contains(nullifier)           // reject replay / double
vote
    assert ZKVerifier.verify(proof, nullifier, claimed_root, proposal_id)
    nullifiers.insert(nullifier)                     // consume nullifier
    return VotingToken { proposal_id: self.proposal_id }
}

// CAST – atomically burns token and registers vote hash
@private
action castVote(
    token: VotingToken,
    vote_hash: Hash                                // H(choice || citizen_nonce) –
irreversible
): VoteReceipt {
    assert token.proposal_id == self.proposal_id
    burn(token)                                     // token destroyed – cannot vote twice
    vote_hashes.append(vote_hash)
    emit VoteRegistered { vote_hash, proposal_id, timestamp: now() }
    return VoteReceipt { vote_hash, proposal_id }
}

// CLOSE – seals round, returns Merkle root for anchoring
@admin
action closeProposal(): MerkleRoot {
    proposalActive = false
    return MerkleTree.root(vote_hashes)
}
}

```

3.3 Token Properties

- **VotingToken has no transfer method. It can only be destroyed via castVote.** Non-transferable:
- **The proposal_id field binds each token to exactly one vote round. Cross-proposal reuse is structurally impossible.** Proposal-scoped:
- **Minting without casting leaves an unspent token. An expiry mechanism on closeProposal burns all unminted tokens automatically.** Atomic mint-cast:
- **VotingToken carries no ADA or DUST. It is a pure capability token.** No monetary value:

3.4 Parliament 3 Contract — Compact Pseudocode

The citizen initiative layer uses a separate contract. Forum content is not vote-sensitive, so it uses lighter privacy — but requires strict accountability to prevent Sybil posting:

```

contract Parliament3 {

    // PseudoID = H(citizen_DID || platform_salt) – unlinkable to real identity
    type PseudoID = Hash

    state topics: Map<TopicID, Topic>
    state used_nullifiers: Set<Hash>    // prevent double-posting identical
content

```

```
    action createTopic(
      pseudoID: PseudoID,
      content_hash: Hash,                // content stored off-chain (IPFS / P3-
      hosted)
      zk_proof: ZKProof                  // proves pseudoID derives from valid
      citizen DID
    ): TopicID {
      assert ZKVerifier.verify(zk_proof, pseudoID)
      // ... store topic, return TopicID
    }

    action upvote(pseudoID: PseudoID, topicID: TopicID, postID: PostID): Unit
  { ... }

  // Trigger government submission – requires 70% top-contributor approval
  action submitToGovernment(
    topicID: TopicID,
    contributor_votes: List<(PseudoID, Bool)>,
    summary_hash: Hash
  ): GovSubmissionReceipt {
    let top_contributors = getTopContributors(topicID)
    let approvals = contributor_votes.filter(v => v._2).length
    assert approvals * 100 / top_contributors.length >= 70
    emit GovSubmission { topicID, summary_hash, participant_count, timestamp:
now() }
    return GovSubmissionReceipt { topicID, summary_hash }
  }
}
```


4. Public Anchor Layer — Cardano Mainnet

Cardano mainnet serves as the immutable, publicly verifiable audit trail for P3. It never receives vote data, citizen identities, or any sensitive information — only Merkle root hashes that allow external parties to verify sidechain integrity.

4.1 Batching & Merkle Anchoring Flow

1. Midnight sidechain accumulates vote hashes in a Merkle tree during each proposal round.
2. At proposal close, the VotingToken contract returns the Merkle root via the closeProposal action.
3. The P3 batcher service picks up the root and constructs a Cardano metadata transaction.
4. Transaction is submitted and confirmed on Cardano mainnet — permanent and tamper-proof.
5. External auditors can independently recompute the root from sidechain data and verify it matches the mainnet anchor.

4.2 Anchor Transaction Metadata Format (CIP-10)

```
{
  "p3_anchor": {
    "version": "1.0",
    "proposal_id": "<uuid>",
    "merkle_root": "<32-byte hex>",
    "vote_count": <integer>,
    "sidechain_block_height": <integer>,
    "timestamp_utc": "<ISO8601>",
    "anchor_type": "vote_round_close"
  }
}
```

4.3 Aiken Anchor Validator

An optional Aiken on-chain validator enforces anchor submission rules — preventing unauthorised parties from injecting false roots:

```
-- Aiken: P3 Anchor Validator (simplified)

type AnchorDatum { authorised_batchers: List<VerificationKey> }
type AnchorRedeemer { merkle_root: ByteArray, proposal_id: ByteArray, sig:
Signature }

validator p3_anchor {
  spend(datum: AnchorDatum, redeemer: AnchorRedeemer, ctx: ScriptContext) {
    let valid_batcher = list.any(datum.authorised_batchers,
      fn(key) { crypto.verify_signature(key, redeemer.merkle_root,
redeemer.sig) })
    let valid_root = bytearray.length(redeemer.merkle_root) == 32
    valid_batcher && valid_root
  }
}
```

}

4.4 Ouroboros PoS & Energy Profile

Cardano's Ouroboros Praos consensus (Proof-of-Stake) means P3 anchoring transactions consume negligible energy compared to PoW alternatives. The Midnight sidechain uses a compatible PoS mechanism. Combined energy footprint of the P3 architecture: comparable to a few households — not an industrial estate.

5. Privacy & Security Model

5.1 Data Classification

Data element	Where it lives	Privacy guarantee
Citizen DID + VC	Citizen device only	Never leaves the wallet. P3 has zero access.
ZKP proof + nullifier	Midnight contract (private execution)	Verified on-chain, nullifier stored. Proof not retained.
Vote hash H(choice nonce)	Midnight sidechain (private)	Stored in Merkle tree. Irreversible. No choice derivable.
Merkle root	Cardano mainnet (public)	No individual vote data derivable from root.
Aggregate results	P3 policy dashboard (public)	Totals only. No individual breakdown.
Pseudonym + avatar (Parliament 3)	P3 platform (public within P3)	Not linkable outside P3. Not linked to DID.

5.2 Threat Model

Threat	Mitigation
Sybil / double vote	Nullifier H(citizen_secret proposal_id) is unique per citizen per proposal. Second proof generates identical nullifier → on-chain rejection.
Vote coercion / receipt forgery	Vote hash H(choice nonce): citizen can generate a plausible false receipt for a different choice using a different nonce. Coercer cannot verify the real vote.
Batcher manipulation	Batcher can only submit valid Merkle roots. Submitting a false root is immediately detectable by any party recomputing from sidechain data.
Midnight node compromise	Private execution environment. Even a compromised node operator cannot extract decrypted vote data without citizen private keys.
Database / API breach	P3 servers store no personal data. A full breach yields no PII. Sidechain stores only vote hashes.
ZKP replay attack	Nullifier is consumed on first use. Replaying the same proof generates the same nullifier → rejected by the nullifiers set in contract state.

5.3 GDPR / AVG Compliance

- **Vote choices stored only as H(choice || nonce). No plain-text political preference is ever persisted. Per GDPR recital 26, irreversibly anonymised data does not constitute personal data.** Article 9 (political opinions — special category):
- **System collects only a ZKP proof and a nullifier per vote. No email, no name, no IP address logged.** Data minimisation (Art. 5(1)(c)):
- **Structurally satisfied — no personal data is held.** Right to erasure (Art. 17):
- **Required before any municipal pilot. A DPIA template is available in the P3 GitHub repository.** DPIA (Art. 35):

6. Integration Surface & API

6.1 Module Overview

Module	Language	Responsibility
identus-bridge	TypeScript / REST	Wraps Hyperledger Identus SDK. Issues VCs, exposes DID resolution, generates ZKP proofs on citizen device.
midnight-contracts	Compact	VotingToken and Parliament3 contracts. Deployed on Midnight testnet → mainnet.
p3-batcher	Haskell / Node.js	Collects vote hashes from sidechain, computes Merkle tree, submits anchor txs to Cardano mainnet.
p3-api	REST + GraphQL	Public API: proposal data, aggregate results, dashboard feeds. No vote-level data exposed.
p3-app	React Native	Citizen-facing mobile app. Embeds Identus wallet SDK for ZKP generation and vote signing.

6.2 Key API Endpoints

```
// Public – no authentication required
GET /api/v1/proposals // list active proposals
GET /api/v1/proposals/:id/results // anonymised aggregate results
GET /api/v1/anchors/:proposal_id // Cardano tx ID + Merkle root for
audit

// Citizen-facing – ZKP-authenticated, no PII transmitted
POST /api/v1/eligibility/verify
  body: { proof: string, nullifier: string, proposal_id: string }
  response: { authorised: bool, token_ref: string }

POST /api/v1/votes/cast
  body: { token_ref: string, vote_hash: string }
  response: { receipt: VoteReceipt }

// Parliament 3
GET /api/v1/parliament-3/topics // list open topics
POST /api/v1/parliament-3/topics // create topic (pseudoID + ZKP)
POST /api/v1/parliament-3/topics/:id/upvote
POST /api/v1/parliament-3/topics/:id/submit // trigger 70% contributor vote
```

7. Technical Roadmap & Open Questions

Phase	Focus	Deliverables
Phase 1 2025–Q2 2026	Architecture & Testnet	Midnight testnet deployment · Identus integration · ZKP circuit design · Compact contract formal audit · DPIA template
Phase 2 Q3 2026–Q1 2027	MVP & Pilot	Cardano preprod anchoring · Municipality VC issuer bridge · React Native app beta · First municipal pilot · Security audit
Phase 3 Q2–Q4 2027	Mainnet	Midnight mainnet migration · Cardano mainnet anchoring · Parliament 3 live · Open API for third-party integrations
Phase 4 2028+	Ecosystem	Hydra L2 for peak load · Cross-municipality federation · eIDAS 2.0 wallet · DAO governance for protocol upgrades

7.1 Open Design Questions — Community Input Welcome

- **BBS+ (wider Identus support, simpler) vs. Groth16 (smaller proof, faster on-chain verification, requires trusted setup). Trade-off: developer friction vs. performance. GitHub discussion open.** ZKP circuit:
- **Immutable contracts maximise trust. Upgradeable proxy pattern introduces admin risk. Current preference: immutable with versioned contract addresses and migration tooling.** Contract upgradeability:
- **Content hashes on-chain, full text off-chain (IPFS or P3-hosted). IPFS maximises decentralisation but complicates moderation. Under discussion.** Parliament 3 content storage:
- **Single batcher is a centralisation risk. Multi-sig batcher committee or DAO-controlled batcher for Phase 3. Contributions welcome.** Batcher decentralisation:

8. Contributing to P3

P3 is actively seeking builders. The project is in pre-MVP phase and welcomes contributions, technical feedback, and co-design proposals across all layers.

Skill area	Open tasks
Compact / Midnight	VotingToken contract refinement · Parliament3 contract · ZKP verifier integration · Testnet deployment scripts
Aiken / Plutus	Anchor validator · Batcher authorisation · On-chain governance (Phase 4)
Hyperledger Identus	Municipality VC issuer bridge · DigiD / eIDAS 2.0 adapter · ZKP circuit implementation
Haskell / Node.js	Batcher service · Merkle tree · Cardano transaction submission

React Native	Citizen app · Identus wallet SDK · Offline-capable ZKP generation
Security / ZK	Circuit audit · Nullifier scheme review · Threat model expansion

Get involved

GitHub: github.com/SBGoodGit/P3

Open issues: github.com/SBGoodGit/P3/issues/new?title=Developer+Interest

Full contract specs and DPIA template are published in the repository.

This whitepaper is a living document — pull requests and technical corrections are welcome.

People Powered Politics

p3democracy.io | github.com/SBGoodGit/P3 | [@P3Democracy](https://twitter.com/P3Democracy)

Part of the GoodCompany Ecosystem · Built on Cardano & Midnight